Computational Data Analysis (02582)

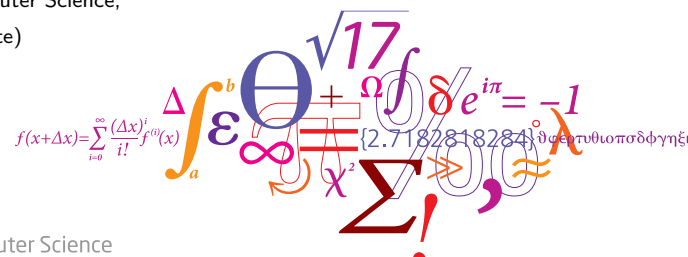
LINEAR AND REGULARIZED CLASSIFICATION

Sneha Das

Assistant Professor

Department of Applied Mathematics and Computer Science,

Technical University of Denmark (DTU Compute)



DTU Compute

Department of Applied Mathematics and Computer Science

Outline I

- Part I: Recap and outstanding questions
- Part II: Linear and regularized classifiers
  - Regularized Classifiers
- Part III: Basis Expansions & Splines
- Summary
- Appendix
  - derivation of LR log-likelihood
  - Derivation of the Augmentation problem to ElasticNet

Last week: Dimensionality

Curse of Dimensionality

- Number of regions grows exponentially with dimensions.
- Data becomes sparse; local neighborhoods are no longer local.

Blessings of Dimensionality

- Correlated features can stabilize predictions.
- Low-dimensional manifolds often exist.
- Underlying Structure can be exploited.

Last week: Regularizations & Multiple Testing

Regularization

- **Methods:** Ridge (L_2), Lasso (L_1), Elastic Net.
- **Effects:** Shrinkage (reduces variance) and sparsity (feature selection).
- **Algorithms:** LARS, Cyclic coordinate descent.

Multiple Testing

- Testing many hypotheses increases the risk of false positives.
- **Bonferroni Corrections:** Controls Family-Wise Error Rate (FWER).
- **Benjamini-Hochberg Corrections:** Controls False Discovery Rate (FDR).

Linear Discriminant Analysis

Classification method

- Based on probability of class belonging - models the posterior probability distribution.
- Linear decision boundary

Generative Classification via Bayes' Theorem

Instead of modeling the decision boundary directly, we model the data distribution for each class.

Using Bayes' Theorem:

$$P(G = k|X = x) = \frac{f_k(x)\pi_k}{\sum_{l=1}^{K} f_l(x)\pi_l}$$

Where:

- $f_k(x) = P(X = x|G = k)$ is the **class-conditional density**.
- $\pi_k = P(G = k)$ is the **prior probability** of class k.

The Multivariate Gaussian Assumption

To use Bayes' rule, we need to choose a functional form for $f_k(x)$. The most common choice is the Multivariate Gaussian density:

$$f_k(x) = \frac{1}{(2\pi)^{p/2}|\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(x-\mu_k)^T \Sigma_k^{-1}(x-\mu_k)\right)$$

Where:

- p is the number of features.
- μ_k is the mean vector for class k.
- Σ_k is the covariance matrix for class k.

Comparing Classes: The Log-Odds

To decide whether a point belongs to class k or class l, we look at the ratio of their posterior probabilities. Taking the logarithm makes the math manageable:

$$\log \frac{P(G = k|X = x)}{P(G = l|X = x)} = \log \frac{f_k(x)}{f_l(x)} + \log \frac{\pi_k}{\pi_l}$$

- If this log-ratio is > 0, we prefer class k.
- If this log-ratio is < 0, we prefer class l.
- The **decision boundary** is the set of points where the ratio equals exactly 0.

The LDA Assumption: Equal Covariances

Crucial LDA Assumption: All classes share the same covariance matrix ($\Sigma_k = \Sigma_l = \Sigma$).
If we plug the Gaussian density into our log-ratio and use a shared Σ:

$$\log \frac{P(G=k|X=x)}{P(G=l|X=x)} = \log \frac{\pi_k}{\pi_l} - \frac{1}{2}(\mu_k + \mu_l)^T \Sigma^{-1} (\mu_k - \mu_l)$$
$$+\mathbf{x^T \Sigma^{-1}} (\mu_\mathbf{k} - \mu_\mathbf{l})$$

Notice: The quadratic terms ($x^T \Sigma^{-1} x$) perfectly cancelled out! The equation is now strictly ***linear*** *in x.*

Linear Discriminant Functions

Because the log-odds are linear, the decision boundaries are straight lines (or hyperplanes). We can define a **Linear Discriminant Function**, $\delta_k(x)$, for each class:

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k$$

Classification Rule: To classify a new observation x, we calculate $\delta_k(x)$ for all classes and assign x to the class with the highest score:

$$\hat{G}(x) = \arg\max_k \delta_k(x)$$

Estimating the Parameters

In practice, we don't know the true means, covariances, or priors. We estimate them directly from the training data:

- **Class Priors:** $\hat{\pi}_k = N_k/N$
- **Class Means:** $\hat{\mu}_k = \sum_{g_i=k} x_i/N_k$
- **Pooled Covariance Matrix:**

$$\hat{\Sigma} = \sum_{k=1}^{K} \sum_{g_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T/(N-K)$$

Quadratic Discriminant Analysis (QDA)

What if the equal covariance assumption is violated?
QDA Assumption:

- $f_k(x)$ is a Multivariate Gaussian density.
- **Unequal Covariances:** Each class has its own distinct covariance matrix (Σ_k).

Consequences:

- The discriminant function becomes quadratic.
- Decision boundaries are conic sections (ellipses, parabolas, hyperbolas).
- **Drawback:** Requires estimating $O(p^2)$ parameters for *each* class, which heavily increases variance in high dimensions.

The High-Dimensional Challenge

The Scenario: You have $N = 100$ patients (50 sick, 50 healthy). You have $p = 10,000$ gene expression levels for each patient.

If you try to use standard LDA or QDA, you must invert the covariance matrix Σ^{-1}.

- The matrix is $10,000 \times 10,000$.
- Because $p \gg N$, the matrix is **singular** (not full rank).
- **Result:** The computer throws an error. The math physically breaks down. We cannot calculate the distances.

We must use Regularization to save the model.

Regularized Discriminant Analysis (RDA)

Proposed by Friedman (1989), RDA builds a bridge between QDA, LDA, and a diagonal covariance matrix.

Option 1: Shrink QDA towards LDA ($\alpha \in [0, 1]$)

$$\hat{\Sigma}_k(\alpha) = \alpha\hat{\Sigma}_k + (1 - \alpha)\hat{\Sigma}$$

Option 2: Shrink the covariance towards its diagonal ($\gamma \in [0, 1]$)

$$\hat{\Sigma}(\gamma) = \gamma\hat{\Sigma} + (1 - \gamma)diag(\hat{\Sigma})$$

Option 3: Shrink LDA towards a spherical matrix ($\gamma \in [0, 1]$)

$$\hat{\Sigma}(\gamma) = \gamma\hat{\Sigma} + (1 - \gamma)\hat{\sigma}^2 I$$

Tuning α and γ via cross-validation allows the data to dictate the optimal flexibility of the decision boundary.

Visualization

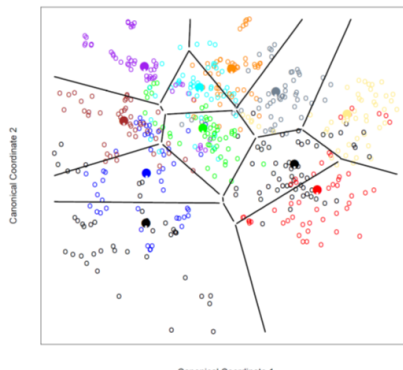
Reduced Rank & Sparse Discriminant Analysis

Reduced Rank Discriminant Analysis (RRDA)

- Projects data into a $K-1$ dimensional subspace that maximizes class separation.
- Excellent for visualization and dimensionality reduction prior to classification.
- *Example:* Projecting a 3D scan of a tumor onto a 1D line that perfectly separates benign from malignant.

Reduced Rank Discriminant Analysis

Classification in a reduced subspace. In higher dimensional subspace, the decision boundaries are hyper-planes and can not be represented as lines. Hence, this technique is very **useful for illustrating class separation**.



The 'Border Guard' Analogy: Discriminative Classification

The Discriminative Approach (Logistic Regression): Act like a border guard.

- The border guard doesn't care *how* fake IDs are manufactured (no modeling of $P(X|G)$).
- They only care about the specific features that separate real from fake (e.g., "Is there a hologram?").
- They draw a line in the sand and model the probability directly: $P(G|X)$.

It makes fewer assumptions about the world, making it safer and more robust when real-world data is messy or non-Gaussian.

Logistic Regression

- It is a Classification method
- Fewer assumptions than LDA
- Linear decision boundary

The Discriminative Approach

Unlike LDA/QDA which model the class distributions $P(X|G)$ (Generative), **Logistic Regression** models the posterior probabilities $P(G|X)$ directly (Discriminative).

For a two-class problem ($Y \in \{0, 1\}$), we use the logistic function:

$$P(Y = 1|X = x) = \frac{e^{\beta_0 + \beta^T x}}{1 + e^{\beta_0 + \beta^T x}}$$

$$P(Y = 0|X = x) = \frac{1}{1 + e^{\beta_0 + \beta^T x}}$$

- This ensures the probabilities strictly sum to 1 and remain between 0 and 1.

The Log-Odds (Logit) Transformation

If we look at the ratio of these probabilities (the odds) and take the logarithm, we get a beautiful linear relationship:

$$\log \frac{P(Y=1|X=x)}{P(Y=0|X=x)} = \beta_0 + \beta^T x$$

Implications:

- **Linear Boundary:** The decision boundary is where $P(Y=1|X) = P(Y=0|X)$, meaning the log-odds equal 0. This is the hyperplane defined by $\beta_0 + \beta^T x = 0$.
- **Interpretation:** Each coefficient β_j represents the change in the *log-odds* of class 1 for a one-unit increase in x_j.

Think-pair-share

Two class problem: Derive expressions for the two class problem
$P(Y = 1|X = x) =?$
$P(Y = 0|X = x) =?$
When

$$\log \frac{P(Y = 1|X = x)}{P(Y = 0|X = x)} = \beta_0 + \beta^T x$$

Likelihood Function

Combine this for all data points x_i:

$$L(\beta_0, \beta) = \prod_{i=1}^{n} P(G = g_{x_i} | X = x_i). \tag{1}$$

Assuming independence, this is the joint probability.

Fitting the Model: Maximum Likelihood (MLE)

The MLE Goal: Find the weights (β) that make our observed training data look as mathematically likely as possible.

- If patient i is sick ($y_i = 1$), we want the predicted $P(x_i)$ to be close to 1.
- If patient i is healthy ($y_i = 0$), we want the predicted $P(x_i)$ to be close to 0.

Instead of multiplying probabilities (which causes computer underflow), we sum the **Log-Likelihood**:

$$\ell(\beta) = \sum_{i=1}^{N} \underbrace{y_i(\beta^T x_i)}_{\text{Rewards a high score if } y=1} - \underbrace{\log(1 + e^{\beta^T x_i})}_{\text{Normalizes it to a valid probability}}$$

Setting the derivative to zero yields no neat algebraic formula. We must use iterative solvers (e.g: Newton-Raphson) to the maximum.

Think-Pair-Share

Context: You fitted a Logistic Regression model to predict lung cancer risk based on X_1 (cigarettes smoked per day).
The model learned the coefficient: $\beta_1 = 0.693$.

Task (3 min): Discuss with your neighbor:

1. Since $e^{0.693} \approx 2.0$, smoking one extra cigarette **doubles the odds** of cancer.
2. *Question:* If a patient's **odds** of cancer double, does their **probability** of getting cancer also double?
3. *Hint:* Think about a patient whose current probability of cancer is $P = 0.60$ (60%).

Vote on Vevox: A) Yes, probability doubles. B) No, probability does not double.

Solution: The Squeeze of the Sigmoid

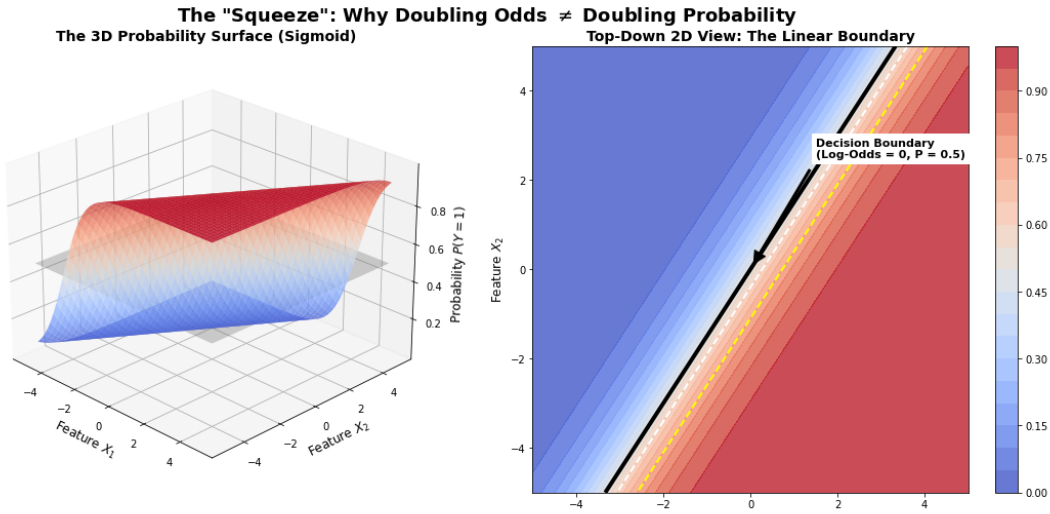
Answer: B (No, probability does not double!)

Why? Probabilities are strictly trapped between 0 and 1. They cannot double infinitely.

- If $P = 0.60$, the Odds are: $\frac{0.60}{1-0.60} = \frac{0.60}{0.40} = 1.5$.
- If we **double the odds** $(1.5 \times 2 = 3.0)$, the new probability is:

$$P_{new} = \frac{Odds}{1 + Odds} = \frac{3.0}{1 + 3.0} = \frac{3}{4} = 0.75$$

The Takeaway: The effect of a coefficient β on the *probability* depends on where you currently are on the Sigmoid curve. It has a massive effect in the middle (50%), but almost no effect at the extremes (99%).


The "Squeeze": Why Doubling Odds ≠ Doubling Probability
The 3D Probability Surface (Sigmoid)
Probability $P(Y=1)$
Feature X_1
Feature X_2
Top-Down 2D View: The Linear Boundary
Decision Boundary
(Log-Odds = 0, P = 0.5)
Feature X_2
Feature X_1
Probability $P(Y=1)$

What about more than two classes?

Multiple (Multinomial) Logistic Regression

How do we classify into $K > 2$ categories (e.g: Healthy, Benign Tumor, Malignant Tumor)?

The Trick: Pick one class to act as the 'Reference' or 'Pivot' (let's say class K). We then model the linear log-odds of every other class *relative* to this pivot:

$$\log \frac{P(G=1|X=x)}{P(G=K|X=x)} = \beta_{10} + x\beta_1$$
$$\log \frac{P(G=2|X=x)}{P(G=K|X=x)} = \beta_{20} + x\beta_2$$
$$\vdots$$
$$\log \frac{P(G=K-1|X=x)}{P(G=K|X=x)} = \beta_{(K-1)0} + x\beta_{K-1}$$

Note: It is completely arbitrary which class we choose as the reference! The predictions will be mathematically identical.

Recovering the Probabilities (The Softmax)

We know that all probabilities must sum to 1: $\sum_{i=1}^{K} P(G = i|X = x) = 1$.

Using this constraint, we can invert the log-odds equations to solve for the actual probabilities:

For the reference class (K):

$$P(G = K|X = x) = \frac{1}{1 + \sum_{i=1}^{K-1} \exp(\beta_{i0} + x\beta_i)}$$

For any other class ($k < K$):

$$P(G = k|X = x) = \frac{\exp(\beta_{k0} + x\beta_k)}{1 + \sum_{i=1}^{K-1} \exp(\beta_{i0} + x\beta_i)}$$

*In deep learning, this exact mathematical formulation is known as the **Softmax** function!*

Regularized Logistic Regression

Few observations (low n) and high dimension (high p) data is a problem also for logistic regression.

One solution is an **elastic net** regularization of the likelihood,

$$
\begin{aligned}
[\beta, \beta_0] &= \arg\max_{\beta_0,\beta} \{\log L(\beta, \beta_0) - P_{\lambda,\alpha}(\beta)\} \\
&= \arg\max_{\beta_0,\beta} \left\{ \sum_{i=1}^{n} \left[y_i(\beta_0 + \beta^T x_i) - \log(1 + e^{1+\beta_0+\beta^T x_i}) \right] - P_{\lambda,\alpha}(\beta) \right\}
\end{aligned}
$$

with

$$
P_{\lambda,\alpha}(\beta) = \lambda \left(\frac{1}{2}(1-\alpha)||\beta||_2^2 + \alpha||\beta||_1 \right)
$$

Use cross-validation for λ and α.

Some properties of LR

- Logistic regression is more robust than LDA (It relies on fewer assumptions)
  When is this a bad thing when compared to LDA?
- Logistic regression handles categorical variables better than LDA
- Observations far away from the boundary are down-weighted
- Breaks down when classes are perfectly separable
- Easy to interpret and explain
- Surprisingly often hard to beat
- Can be combined with regularization of parameters ($n < m$)
- Can be generalized to multi-class problems

Logistic Regression vs. LDA

Both methods result in strictly **linear decision boundaries**, but they arrive there differently:

- **LDA (Generative):**
  - Assumes features X are Multivariate Gaussian.
  - Uses all data points to estimate μ and Σ.
  - More stable if assumptions hold, especially with small N.
- **Logistic Regression (Discriminative):**
  - Makes NO assumptions about the distribution of X.
  - Focuses primarily on the data points near the decision boundary.
  - Generally safer and more robust in messy, real-world data.

Think-Pair-Share

Context: You are predicting if a bank transaction is Fraud or Legit. You fit both LDA and Logistic Regression. Their decision boundaries are almost identical.

Suddenly, a massive **outlier** appears in the 'Legit class. It is obviously a valid transaction, but its value is $10 Million (it sits far away from the decision boundary).

Task (3 min): Discuss with your neighbor:

1. How will the **LDA** decision boundary react to this outlier?
2. How will the **Logistic Regression** boundary react?

Vote on Vevox: A) Both shift. B) LDA shifts, LR ignores it. C) LR shifts, LDA ignores it.

Solution: Global Profiler vs. Local Border Guard

Answer: B (LDA shifts dramatically, LR barely moves).

Why the math behaves this way:

- **LDA (The Profiler):** Calculates the mean (μ) and variance (Σ) of the *entire* class. A massive outlier physically pulls the mean, altering the statistical profile and shifting the boundary.
- **LR (The Border Guard):** The outlier is already far on the "Legit" side. The model evaluates its probability as $P \approx 0.9999$. Moving it further away doesn't change the logistic loss, so the border guard ignores it!

Visualization

Supervised Classification Overview

Overview of classification methods - so far.

Method	$p > n$	Dimension Reduction	Variable Selection	Weights on Observations	Assuming Independence	Distributional Assumptions	Equal Covariances	Non-linear Decision Boundary
Linear DA (LDA)						x	x	
Reduced-rank LDA		x				x	x	
Quadratic DA (QDA)						x		x
Logistic Regression				x				
K-Nearest Neighbor (KNN)				x				x
Regularized DA	x	x	x		(x)	x	x	
Sparse DA	x	x	x			x	x	

The Non-Linear Reality

Linear models (LDA, Logistic Regression) assume a constant rate of change. But the real world is rarely strictly linear.

Real-World Examples:

- **Medicine:** A drug's effectiveness isn't linear. Too little does nothing, the right amount cures, too much is toxic.
- **Audio Frequencies:** Human hearing sensitivity peaks in the middle frequencies and drops off at the extremes.

The Goal: How do we capture these non-linear realities without abandoning our stable, well-understood linear math?

$\rightarrow$ *We transform the features X into a new space $h(X)$ where the relationship **is** linear.*

Transform to be able to use a Linear Model

Idea: replace variables (columns) of the data matrix, X, with transformations $h(X)$

The linear model

$$y = X\beta = \sum_{i=1}^{p} \beta_i x_i \rightarrow \sum_{i=1}^{M} \beta_i' h_i(X)$$

In this way we handle non-linear problems with our well known linear models

Basis Expansion

- We are not limited to use our data as they are
- Linear models
  - Easy to interpret
  - First order Taylor expansion of non-linearities
  - Might be ok even for non-linear data if we have few observations
- Non-linear problem - transform data and use linear model
  - $h_m(X) = X_j^2$ and $h_m(X) = X_j X_k$
  - $h_m(X) = \log(X_j)$ or $h_m(X) = sqrt(X_j)$
  - $h_m(X) = \frac{X - m_X}{s_X}$ (always used when using regularization)
  - $h_m(X_{(i)}) = i$, sort data $X_{(1)} \leq X_{(2)} \leq ...$ and use the rank
  - Either replacing X with $h_m(X)$ or expanding $\{X, h_m(X)\}$

First Attempts: Polynomials & Step Functions

1. Global Polynomials ($X, X^2, X^3, \dots$)

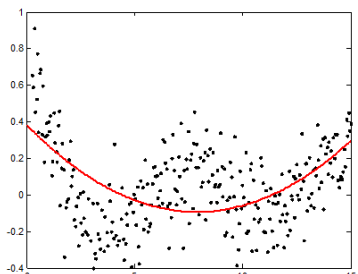
- **The Flaw:** They are *global*. If you tweak a high-degree polynomial to fit a data point in Denmark, the curve might wildly fluctuate in Australia. They extrapolate terribly.

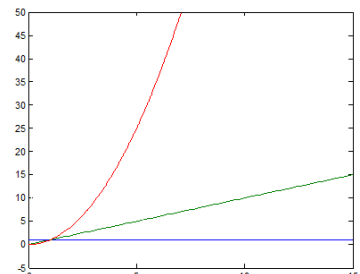
2. Step Functions (Binning)

- **The Idea:** Cut the X-axis into discrete bins and fit a constant in each.
- **The Flaw:** They are *discontinuous*. A person aged 39.9 gets a completely different prediction than a person aged 40.0. This makes no physical sense.

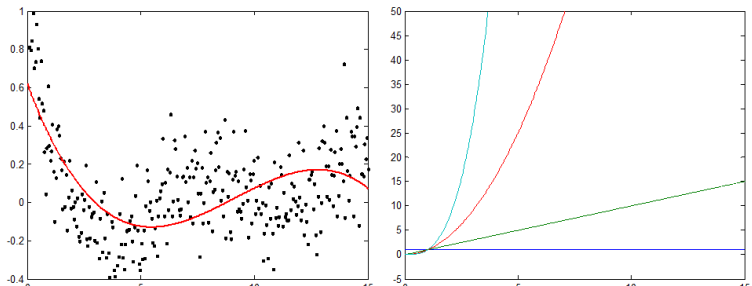
Example: Let's Try With Up to X^2

```
X = [ones(n,1) x x.^2];
y = X*(X\y)
```

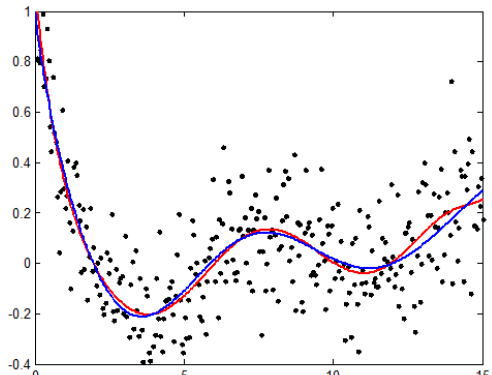




Example: Let's Try With Up to X^3

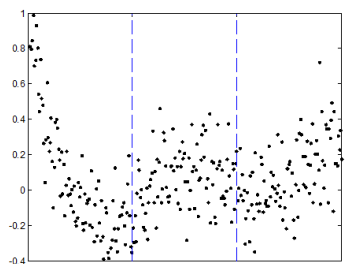


Example: Let's Try With Up to X^8



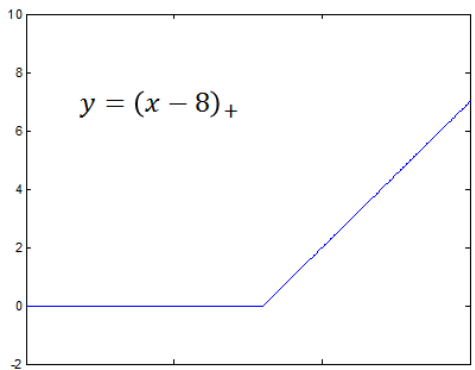
Piece-Wise

- To introduce flexibility while keeping the variance under control, we define **different basis functions for different intervals of x.**
- Example, divide the range of x in three parts



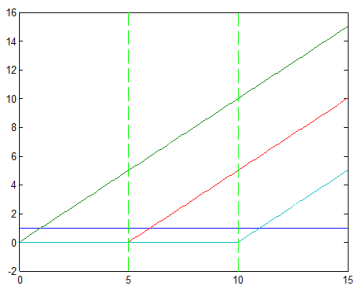
This can be achieved using the Hinge function

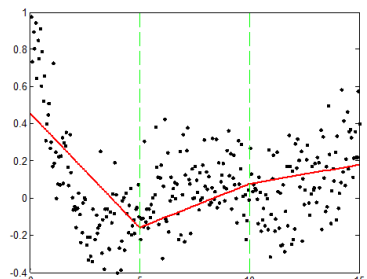
- Introducing the "**hinge**" function $y = (f(x))_+$
- Zero when $f(x)$ is less than zeros, otherwise $f(x)$



Example: Try with Piece-Wise Linear Functions

```
X = [ones(n,1) x max(0,x-5) max(0,x-10)];
```





First Attempts: Polynomials & Step Functions

1. Global Polynomials ($X, X^2, X^3, \dots$)

- **The Flaw:** They are *global*. If you tweak a high-degree polynomial to fit a data point in Denmark, the curve might wildly fluctuate in Australia. They extrapolate terribly.

2. Step Functions (Binning)

- **The Idea:** Cut the X-axis into discrete bins and fit a constant in each.
- **The Flaw:** They are *discontinuous*. A person aged 39.9 gets a completely different prediction than a person aged 40.0. This makes no physical sense.

The Solution: Piecewise Polynomials (Splines)

The Idea: Divide the domain into regions separated by points called **Knots** (ξ). Fit a simple polynomial (e.g.: cubic) in each region.

To avoid the 'Step Function' problem, we enforce strict **Constraints at the Knots**:

1. The pieces must touch (Continuous).
2. The slopes must match (Continuous 1st derivative).
3. The curve's 'bend' must match (Continuous 2nd derivative).

*When we apply these constraints to piecewise cubic polynomials, we get a **Cubic Spline**.*

Visualization

Cubic Splines

Splines are piece-wise polynomials

- The basis functions are

```
X = [ones(n,1) x x.^2 x.^3 max(0,x-5).^3 max(0,x-10).^3
```

- Or in the proper way

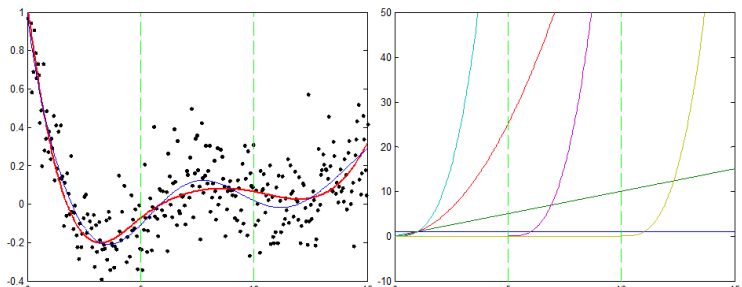
$$h_0(x) = 1$$
$$h_1(x) = x$$
$$h_2(x) = x^2$$
$$h_3(x) = x^3$$
$$h_4(x) = (x-5)_+^3$$
$$h_5(x) = (x-10)_+^3$$

- Cubic splines have continuous first and second derivatives at the knots.
  - I.e a smooth function

Example: Try With Cubic Splines

Spline approximation is

$$f(t) = b_0 + b_1 t + b_2 t^2 + b_3 t^3 + b_4 (t-5)_+^3 + b_5 (t-10)_+^3$$



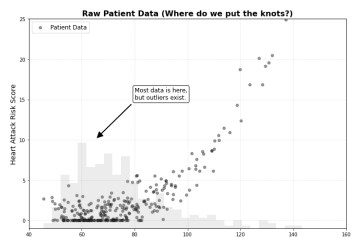
Notice that this non-linear function was obtained with a linear model

Think-Pair-Share: The Knot Dilemma

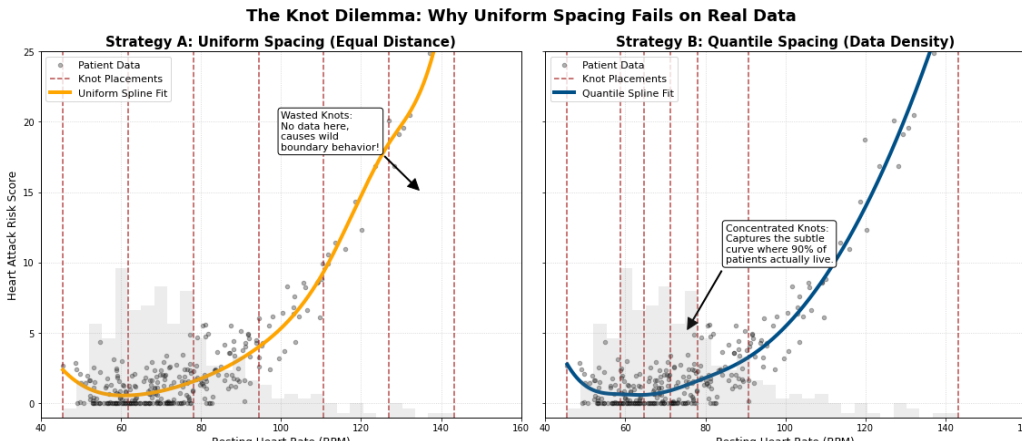
Context: You are modeling a patient's risk of heart attack based on their resting heart rate (BPM). The relationship is non-linear. You decide to use a Cubic Spline.

Task (3 min): Discuss with your neighbor:

1. Where would you place the **knots** along the BPM axis? (e.g., 40, 60, 100, 150?)
2. Should knots be spaced evenly across the axis, or should they be clustered in specific areas? Why?



Go to Vevox and enter your knot placement strategy...


The Knot Dilemma: Why Uniform Spacing Fails on Real Data
Strategy A: Uniform Spacing (Equal Distance)
Patient Data
Knot Placements
Uniform Spline Fit
Wasted Knots: No data here, causes wild boundary behavior!
Heart Attack Risk Score
Resting Heart Rate (BPM)
Strategy B: Quantile Spacing (Data Density)
Patient Data
Knot Placements
Quantile Spline Fit
Concentrated Knots: Captures the subtle curve where 90% of patients actually live.
Resting Heart Rate (BPM)

Solution: Domain Knowledge vs. Data Density

How do we choose knots in practice?

- **Data Density (Quantiles):** Place knots where the data is! If 90% of patients have a heart rate between 60 and 90, put more knots there to capture the subtle variations. (Don't waste parameters on 150+ BPM if you only have 2 patients there).
- **Domain Knowledge:** Place knots at known medical thresholds. (e.g., Clinical guidelines define Tachycardia at > 100 BPM, so a knot at 100 makes biological sense).
- **Degrees of Freedom:** More knots = more flexibility = higher risk of overfitting. We often use Cross-Validation to choose the total *number* of knots.

Why Cubic? The Rollercoaster Analogy

Think of a spline as designing a track for a rollercoaster. The 'knots' are where different segments of steel track are welded together.

- **Continuous (0th Deriv):** The tracks connect. (You don't fall off).
- **1st Derivative:** No sharp corners. (You don't break your neck).
- **2nd Derivative:** Smooth changes in acceleration. (You don't throw up).

Why stop at Cubic (X^3)? Cubic is the lowest mathematical degree that gives us enough parameters to control the 0th, 1st, and 2nd derivatives. Furthermore, the human eye cannot detect discontinuities in the 3rd derivative. It looks flawlessly smooth!

The Math: Truncated Power Basis

How do we feed this to a Logistic Regression model? We create new feature columns.

For a cubic spline with two knots at $x = 2$ and $x = 5$, our linear log-odds equation becomes:

$$\log \frac{P(G = 0|X = x)}{P(G = 1|X = x)} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - 2)_+^3 + \beta_5 (x - 5)_+^3$$

The 'Hockey Stick' function: $(x - \xi)_+^3$

- If $x \leq \xi$, the value is exactly 0. (It has no effect before the knot).
- If $x > \xi$, it adds a new cubic trajectory.

*Notice: The equation is non-linear in x, but perfectly **linear in** β. We just solve it using standard logistic regression!*

Natural Cubic Splines: Taming the Tails

The Problem: Standard cubic splines behave erratically at the boundaries (before the first knot and after the last knot). Since there are fewer data points at the extremes, the X^3 terms can shoot off to infinity.

The Fix: Natural Cubic Splines

- We add an extra mathematical constraint: The function must be strictly **linear** beyond the boundary knots.
- This forces the 2nd and 3rd derivatives to be 0 at the boundaries.
- **Benefit:** It prevents wild extrapolation and frees up 'degrees of freedom' (parameters) that we can spend on adding more knots in the middle where the data actually lives!

Summary: Linear Classifiers

Linear Discriminant Analysis (LDA)

- Linear decision boundary.
- Relies on Gaussian and equal covariances assumptions.
- Non-equal gave QDA and non-linear boundary.
- $\rightarrow$ *Regularized versions (RDA, SDA, RRDA).*

Logistic Regression

- Linear decision boundary with fewer assumptions than LDA.
- Coefficients can be directly interpreted (odds ratios).
- $\rightarrow$ *Regularized versions (L_1, L_2).*

Basis Expansions

- Gives linear methods non-linear decision boundaries (e.g., cubic splines).

Fitting the Model: Maximum Likelihood (MLE) for logistic regression

The MLE Goal: Find the weights (β) that make our observed training data look as mathematically likely as possible.

- If patient i is sick ($y_i = 1$), we want the predicted $P(x_i)$ to be close to 1.
- If patient i is healthy ($y_i = 0$), we want the predicted $P(x_i)$ to be close to 0.

Instead of multiplying probabilities (which causes computer underflow), we sum the **Log-Likelihood**:

$$\ell(\beta) = \sum_{i=1}^{N} \underbrace{y_i(\beta^T x_i)}_{\text{Rewards a high score if } y=1} - \underbrace{\log(1 + e^{\beta^T x_i})}_{\text{Normalizes it to a valid probability}}$$

Unlike OLS, setting the derivative to zero yields no neat algebraic formula. We must use iterative solvers (e.g., Newton-Raphson) to "climb the hill" to the maximum.

Derivation (1/4): The Bernoulli Distribution

For a single observation i, the binary outcome $y_i \in \{0, 1\}$ follows a Bernoulli distribution.

Let $p_i = P(y_i = 1|x_i)$. The probability of predicting the correct observed class for patient i can be written compactly as:

$$P(y_i|x_i) = p_i^{y_i}(1 - p_i)^{1-y_i}$$

Notice how this acts like a switch based on the actual label:

- If $y_i = 1$, the exponent $(1 - y_i)$ becomes 0, leaving just p_i.
- If $y_i = 0$, the exponent y_i becomes 0, leaving just $1 - p_i$.

Derivation (2/4): The Logistic Function

In logistic regression, we model the probability p_i using the sigmoid/logistic function:

$$p_i = \frac{e^{\beta^T x_i}}{1 + e^{\beta^T x_i}}$$

To complete our formula, we also need the probability of the negative class ($y_i = 0$), which simplifies nicely:

$$\begin{aligned} 1 - p_i &= 1 - \frac{e^{\beta^T x_i}}{1 + e^{\beta^T x_i}} \\ &= \frac{1 + e^{\beta^T x_i} - e^{\beta^T x_i}}{1 + e^{\beta^T x_i}} \\ &= \frac{1}{1 + e^{\beta^T x_i}} \end{aligned}$$

Derivation (3/4): The Likelihood Function

Assuming our N patients are independent, the joint probability of seeing *all* our training data is simply the product of their individual probabilities. This is the **Likelihood Function**, $L(\beta)$:

$$L(\beta) = \prod_{i=1}^{N} p_i^{y_i}(1-p_i)^{1-y_i}$$

To prevent computer underflow from multiplying many tiny decimals, we take the natural logarithm to convert the product into a sum, yielding the **Log-Likelihood**, $\ell(\beta)$:

$$\ell(\beta) = \log L(\beta) = \sum_{i=1}^{N} \Big[y_i \log(p_i) + (1-y_i)\log(1-p_i) \Big]$$

Derivation (4/4): Simplifying to the Final Form

Substitute the fractions we derived for p_i and $1 - p_i$:

$$\ell(\beta) = \sum_{i=1}^{N} \left[y_i \log\left(\frac{e^{\beta^T x_i}}{1 + e^{\beta^T x_i}} \right) + (1 - y_i) \log\left(\frac{1}{1 + e^{\beta^T x_i}} \right) \right]$$

Apply logarithm rules ($\log(A/B) = \log A - \log B$ and $\log 1 = 0$):

$$\ell(\beta) = \sum_{i=1}^{N} \left[y_i(\beta^T x_i - \log(1 + e^{\beta^T x_i})) + (1 - y_i)(-\log(1 + e^{\beta^T x_i})) \right]$$

Expand and gather the terms:

$$\ell(\beta) = \sum_{i=1}^{N} \left[y_i \beta^T x_i \underbrace{- y_i \log(1 + e^{\beta^T x_i}) - \log(1 + e^{\beta^T x_i}) + y_i \log(1 + e^{\beta^T x_i})}_{\text{The } y_i \log(\dots) \text{ terms cancel out completely}} \right]$$

$$= \sum_{i=1}^{N} \left[y_i(\beta^T x_i) - \log(1 + e^{\beta^T x_i}) \right]$$

1. The Elastic Net Objective

The Elastic Net combines both L_1 (Lasso) and L_2 (Ridge) penalties to achieve both sparsity and grouped selection. The objective function to minimize is:

$$J(\beta) = \underbrace{\|y - X\beta\|_2^2}_{\text{Residual Sum of Squares}} + \underbrace{\lambda_2\|\beta\|_2^2}_{\text{Ridge Penalty}} + \underbrace{\lambda_1\|\beta\|_1}_{\text{Lasso Penalty}}$$

The Goal: We have excellent, highly-optimized solvers (like LARS or Coordinate Descent) for the standard LASSO problem. Can we trick a LASSO solver into solving the Elastic Net problem simply by modifying our input data?

2. Naïve Augmentation: Construction

Step 1: We "hide" the L_2 penalty inside the data by appending p artificial rows (where p is the number of predictors).

We define the augmented matrices X_{aug} and y_{aug}:

$$X_{aug} = \begin{pmatrix} X \\ \sqrt{\lambda_2} I_p \end{pmatrix}, \quad y_{aug} = \begin{pmatrix} y \\ \mathbf{0}_p \end{pmatrix}$$

- X_{aug} is now an $(n+p) \times p$ matrix.
- I_p is a $p \times p$ identity matrix.
- y_{aug} is an $(n+p) \times 1$ vector padded with zeros.
- $\sqrt{\lambda_2}$ regulates the strength of the Ridge influence.

3. Naïve Augmentation: The Absorption

Let's calculate the standard Residual Sum of Squares (RSS) using our new augmented data:

$$\|y_{aug} - X_{aug}\beta\|_2^2 = \left\| \begin{pmatrix} y \\ \mathbf{0}_p \end{pmatrix} - \begin{pmatrix} X \\ \sqrt{\lambda_2} I_p \end{pmatrix} \beta \right\|_2^2$$

$$= \left\| \begin{pmatrix} y - X\beta \\ \mathbf{0}_p - \sqrt{\lambda_2} I_p \beta \end{pmatrix} \right\|_2^2$$

Because the norm of a block vector is the sum of the norms of its blocks:

$$\|y_{aug} - X_{aug}\beta\|_2^2 = \underbrace{\|y - X\beta\|_2^2}_{\text{Original RSS}} + \underbrace{\|\sqrt{\lambda_2}\beta\|_2^2}_{\text{The Ridge Penalty!}}$$

$$= \|y - X\beta\|_2^2 + \lambda_2 \|\beta\|_2^2$$

4. The Equivalent LASSO Problem

By substituting our finding back into the original Elastic Net equation, the L_2 term is completely absorbed into the error term.

Only the L_1 penalty remains visible:

$$\min_{\beta} \|y_{aug} - X_{aug}\beta\|_2^2 + \lambda_1 \|\beta\|_1$$

Crucial Implication

- We can feed (X_{aug}, y_{aug}) directly into a standard LASSO solver.
- The solver is 'blind' to the hybrid penalty; it thinks it's just doing LASSO.

5. The "Double Shrinkage" Problem

However, the "Naïve" solution we just derived is actually biased. It suffers from a phenomenon called **double shrinkage**:

1. **L2 Shrinkage:** The augmented data X_{aug} implicitly applies a Ridge penalty, pulling the coefficients toward zero.
2. **L1 Shrinkage:** The Lasso solver then applies the L_1 penalty on top of those *already-shrunk* coefficients.

The Result

The coefficients become 'too small', leading to unnecessarily high bias. We have effectively 'taxed' the coefficients twice.

6. The Rescaled Augmentation (The Fix)

To fix double shrinkage, we introduce a scaling factor: $c = (1 + \lambda_2)^{-1/2}$. We define our final extended data X^* and y^* as:

$$X^* = \frac{1}{\sqrt{1+\lambda_2}} \begin{pmatrix} X \\ \sqrt{\lambda_2} I_p \end{pmatrix} \quad \text{and} \quad y^* = \begin{pmatrix} y \\ \mathbf{0}_p \end{pmatrix}$$

Notice that $X^* = cX_{aug}$, but y^* remains unscaled ($y^* = y_{aug}$).

Let's prove why this exact scaling factor perfectly corrects the Ridge shrinkage.

7. Proving the Rescaling Works

Let's look at the unpenalized OLS solution using our rescaled X^* and y^*. The normal equations are $(X^{*T}X^*)\beta^* = X^{*T}y^*$.

First, calculate $X^{*T}X^*$ and $X^{*T}y^*$:

$$X^{*T}X^* = \left(\frac{1}{\sqrt{1+\lambda_2}}\right)^2 \begin{pmatrix} X^T & \sqrt{\lambda_2}I_p \end{pmatrix} \begin{pmatrix} X \\ \sqrt{\lambda_2}I_p \end{pmatrix} = \frac{1}{1+\lambda_2}(X^TX + \lambda_2 I_p)$$

$$X^{*T}y^* = \frac{1}{\sqrt{1+\lambda_2}} \begin{pmatrix} X^T & \sqrt{\lambda_2}I_p \end{pmatrix} \begin{pmatrix} y \\ \mathbf{0}_p \end{pmatrix} = \frac{1}{\sqrt{1+\lambda_2}} X^Ty$$

8. Proving the Rescaling Works (Cont.)

Substitute these back into the normal equations:

$$\frac{1}{1+\lambda_2}(X^T X + \lambda_2 I_p)\beta^* = \frac{1}{\sqrt{1+\lambda_2}} X^T y$$

Multiply both sides by $\sqrt{1+\lambda_2}$:

$$\frac{1}{\sqrt{1+\lambda_2}}(X^T X + \lambda_2 I_p)\beta^* = X^T y$$

Rearranging this, we find:

$$\frac{1}{\sqrt{1+\lambda_2}}\beta^* = (X^T X + \lambda_2 I_p)^{-1} X^T y$$

Conclusion

The right side is the exact formula for standard Ridge Regression ($\hat{\beta}_{ridge}$). Therefore, $\beta^* = \sqrt{1+\lambda_2}\hat{\beta}_{ridge}$. The scaling factor exactly inflates the coefficients to undo the double shrinkage, allowing algorithms like LARS to trace the true Elastic Net path!